Step 3 — Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — The rate limiter
- Hint 2 — The container, and keeping the key out of it

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The rate limiter

Each run costs five agents' worth of model calls, so a caller in a loop gets expensive fast. Keep a record of **when** recent requests came in:

- on each request, keep only the timestamps from the last 60 seconds;
- if 5 or more are left, raise a **429**;
- otherwise add this request's time and carry on.

Check it **first** in `POST /run-task`, before anything costs money. A "rolling window" like this can't be beaten at a reset boundary, the way a counter that resets every minute can.

Doc12's Build Task limiter, in `routes.py`:

- `RATE_LIMIT = 5`, `RATE_WINDOW_SECONDS = 60`, `request_times = {}` (caller → list of timestamps).
- `check_rate_limit(caller_key)` — a loop that keeps `t` where `now - t < RATE_WINDOW_SECONDS`, then the 429 check.
- Call it with `http_request.client.host` — one bucket **per caller**, so one noisy caller can't block everyone.
- Write the limitation in a comment next to the dict: memory only, reset on restart, and each container counts separately.

In the tests, each test that posts must start with a fresh `request_times` — `monkeypatch.setattr(routes, "request_times", {})` in `use_fake_team()` — and the 429 test can set `RATE_LIMIT` to 2.

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The container, and keeping the key out of it

A `Dockerfile` is the recipe: start from `python:3.11-slim`, copy `requirements.txt` and install it, copy the code, install the package, and start uvicorn.

The key must **never** be inside the image. It's passed when the container starts: `docker run --env-file .env ...`. Anything copied into an image stays readable in its layers.

Also check the failure: `docker run` with no key should stop at once with a clear error, not start a half-working API.

```
Dockerfile:
    FROM python:3.11-slim / WORKDIR /app
    COPY requirements.txt .  ->  RUN pip install -r ...
    COPY . .                 ->  RUN pip install -e .
    EXPOSE 8000
    CMD uvicorn contentforge.api.main:app --host 0.0.0.0 --port 8000

.dockerignore: .env, .git, .venv/, __pycache__/, runs.db, ...
```

- Requirements **before** `COPY . .` — a code edit then skips the slow install.
- `--host 0.0.0.0` — the default `127.0.0.1` can't be reached from outside the container.
- Prove it: `docker history --no-trunc contentforge-pro | grep "sk-"` must print nothing.
- The startup check is already there: `load_config()` in `api/main.py` raises `MissingConfigError`.

Full solution: Show me the solution